

Statistics Component - Technical Document

Written by Karlo Sadural

System Overview

This document outlines the creation and implementation of a modular statistics component in Unreal Engine 5.5.4 using a combination of C++ and Blueprint Visual Scripting. This system holds pertinent information crucial to all players and enemies inside the game. This includes several variables, denoted below with their variable type:

- Current Health (float)
- Maximum Health (float)
- Current Mana (float)
- Maximum Mana (float)
- Mana Level (float)
- Strength (int)
- Dexterity (int)
- Arcane (int)
- Charisma (int)
- Loot (int)
- XP (int)
- Health Level (int)
- Strength Level (int)
- Charisma Level (int)
- Dexterity Level (int)
- Agility Level (int)
- Endurance Level (int)

1. The Struct

First, the structure needed to be defined to hold these variables. An actor class, *S_CharStats.cpp* (and, in parallel, *S_CharStats.h*) was defined. Inside the header file, a **struct** was defined by the name `FCharStats`. This held the denoted variables.

The rationale behind this decision was to decouple it from the default pawn class. Since enemies will also need an amount of health and mana, we'll be able to assign them with this structure as opposed to having to set up these variables each time we create a new enemy class.

2. The Component

Following the structure's creation, an actor component, *StatsComp.cpp* (and, in parallel, *StatsComp.h*), was created, for the sake of simplicity. A technical designer could then simply add this component to any Blueprint class in-engine.

This class holds the defined struct, *FCharStats*, which, as previously stated, held the several pertinent variables.

Since we're also developing a multiplayer game, on initialization, it is set to replicate across clients. In other words, each client has their own *StatsComp*, and consequently, their own amount of Current Health, Max Health, etc, etc.

Key Functions:

- RetrieveFinalHealth()
- RetrieveFinalMana()
- IsDead()
- ResourceCheck()
- RegenerateHealth()
- HandleHealthRegenTick()

Retrieve Final Health

Our programming team lead wrote a universal Function Library. One of these functions inside that Function Library is *GetFinalHealth()*, which factors in bonuses to a player's health score. In essence, the function Retrieve Final Health simply **scales the player's maximum health** by factoring in bonuses from:

- Health level
- Strength
- Max Health
- Current Health
- Endurance level

Retrieve Final Mana

Similar to above, but instead, this function relies on a function written in the Function Library called *GetFinalMana()*. This also scales with bonuses affecting player stats. These bonuses are:

- Current mana
- Max mana
- Mana level
- Arcane

- Charisma

Is Dead

IsDead() is a simple bool const function, which returns true if CharStats.CurrentHealth $\leq$ 0, while it returns false if CharStats.CurrentHealth $>$ 0.

Resource Check

ResourceCheck() is a bool const function that takes two inputs:

- Resource, which is an enumeration and can be either Health or Mana
- RequestedAmt, which is a float

If the Resource being checked is Health, the function compares the pawn's Current Health to the Requested Amount. It returns true if it is greater than the requested amount, returning false if it is not.

If the Resource being checked is Mana, the function compares the pawn's Current Mana to the Requested Amount. It returns true if it is greater than the requested amount, returning false if it is not.

Regenerate Health

RegenerateHealth() is a function that takes two inputs:

- Amount, which is a float
- Duration, which is a float

Additional components are needed for this function, which are:

- FTimerHandle RegenTimerHandle; a Timer Handle for the Regeneration function
- RegenRate, which is a float
- RegenPerTick, which is a float
- RemainingRegen, which is a float

When this function is called, the user should define numbers for both Amount and Duration. In the event that either one is set to zero (e.g., the designer/programmer forgot to set this number), the function simply returns and does not run. However, if this is false, the function proceeds.

We then find the World Timer Manager, which is done by `GetWorld()->GetTimerManager()`, because this is a component class instead of an actor class. For safety, if the defined timer *RegenTimerHandle* is running (**aka the player is already healing**) we clear the timer and force the function to stop before the program proceeds.

We then take in the Amount, which the designer defines, and assign it to our earlier variable, *RemainingRegen*. *RegenRate* simply becomes a small value, 0.1, for example, and this is how much is added to the player's health when called.

Finally, before the actual regeneration happens, we set our earlier variable, *RegenPerTick*, to the designer's defined $(\text{Amount} / \text{Duration}) * [\text{the given Regen Rate}]$. We are finally able to start regenerating.

In the event that the player's current health is less than their max health, the regeneration is allowed to run. If the player's current health $\geq$ max health, the regeneration simply will not happen.

Then, using our early defined timer handle, *RegenTimerHandle*, we set a Timer on the below function, *HandleHealthRegenTick*. The reason we use a timer is that, one, we want regeneration to happen gradually (over time), and two, we want the function to loop.

Handle Health Regen Tick

HandleHealthRegenTick() is where the actual gradual increasing of the player's health occurs—in other words, the **actual regeneration**.

First, we need to understand how much is added each time the function ticks/runs. To do this, we create a variable to track this, a:

- Float called *ActualRegen*

And then, we clamp this float between the earlier defined variables, *RegenPerTick*, and *RemainingRegen*.

Then, we add to the player's health. For healing, we simply clamp the player's current health using these three values:

- Value: $\text{Current Health} + \text{ActualRegen}$
- Minimum Value: 0.0f
- Maximum Value: *MaxHealth* (to prevent going over their maximum health)

After this, we subtract *RemainingRegen* - *ActualRegen* to track how much more needs to be added to the player's health, so we know how close we are to finishing healing the player.

The program understands it is done running the player and no longer needs to regenerate their health when one of these two conditions is met:

- *RemainingRegen* is $\leq 0.0f$. This is because we've reached the amount we're meant to heal the player by, so we know we're done. For example, if we want to heal the player 30, by the time the player has healed 30 hit points, *RemainingRegen* = $0.0f$.
- *CurrentHealth* $\geq$ *MaxHealth*. Implemented again here just for safety and to prevent player from having more than their maximum health.

Future Iterations/Improvements

An important feature to add to this would perhaps be a debug feature, creating a command that hard-sets a variable to a value, such that a developer could properly test a function.

Future iterations include connecting this to a UI via a Widget Blueprint. For testing purposes, a simple widget was created which simply had a text block bound to however much the player's health was. When the regeneration function was run, that text properly updated. However, this is rudimentary and needs to be tied to a proper health bar; in the future with this system, this would be implemented as well.

Considerations, Closing Thoughts

In truth, the pawn class simply could have been given the struct as opposed to a component. Or the struct could be defined as a variable in the player blueprint, however, this seemed more modular and decoupling. By having the component handle all the statistical calculations, we keep the player blueprint clean and are able to make the code modular, saving time so that we don't have to rewrite functions over and over again.